

CSCC 全国大学生计算机系统能力大赛



Yat-CASched

（Proj59 ： Linux 内核低时延调度器）

——面向多核系统的缓存感知低时延调度技术及实现

• **队伍名称**：从容应队

• **答辩人**：林炜东、马福泉、刘昊

• **队伍编号**：T202510558995172

0 队伍介绍

ABOUT US

我们队伍名称为**从容应队**，

队伍编号：T202510558995172 ，

来自中山大学RTS-SYSU实验室，

基本情况如下：

1. 队长**林炜东**，中山大学计算机学院计算机科学与技术专业二年级本科生。

2. 队员**马福泉**，中山大学计算机学院计算机科学与技术专业二年级本科生。

3. 队员**刘昊**，中山大学计算机学院计算机科学与技术专业二年级本科生。

0 队伍介绍



校内指导老师简介：

赵帅，副教授，教育部首批海外引才专项、广东省珠江学者青年拔尖人才、中山大学“百人计划”引才专项获得者。主要从事实时系统、操作系统领域的理论与应用研究，旨在围绕操作系统，为上层应用提供高性能、硬实时的计算与通讯保障，专注于复杂实时系统设计与分析、系统资源共享与管理、硬软件协同设计与寻优等具体方向。

个人主页https://cse.sysu.edu.cn/content/6598

所选赛题：Proj59: Linux 内核低时延调度器

项目导师：谢秀奇、成坚

赛题链接：https://github.com/oscomp/proj59-linux-fast-scheduler

目录

CSCC 全国大学生计算机系统能力大赛

中山大學 SUN YAT-SEN UNIVERSITY

1 背景和动机 BACKGROUNDS & MOTIVATION

2 项目介绍 PROJECT INTRODUCTION

3 总体架构 OVERALL ARCHITECTURE

4 设计与实现 DESIGN & IMPLEMENTATION

5 项目测试 PROJECT TESTING

6 总结展望 CONCLUSION & OUTLOOK

1 背景和动机

BACKGROUNDS & MOTIVATION



赛题背景

- 随着多核处理器技术的发展，现有 **Linux 调度器在特定场景下的局限性**日益凸显。
- 当前 Linux 内核中的**完全公平调度器 (CFS)** 主要关注**任务公平性和负载均衡**，但在设计时并未充分考虑现代多核处理器的**缓存层次结构特征**。
- 现有调度器**(CFS)** 存在**三个核心问题**：调度器**结构日益臃肿**，代码复杂度持续上升，**调度开销增加**； **缺乏对关键线程的优先调度支持**，无法有效识别和处理重要任务；**时延不可控**，在高并发场景下任 务调度延迟难以预测，严重影响系统响应性能。



设计目标

针对 CFS 的问题，我们设计的Yat_CAsched调度策略

目标是具备以下四大优势：

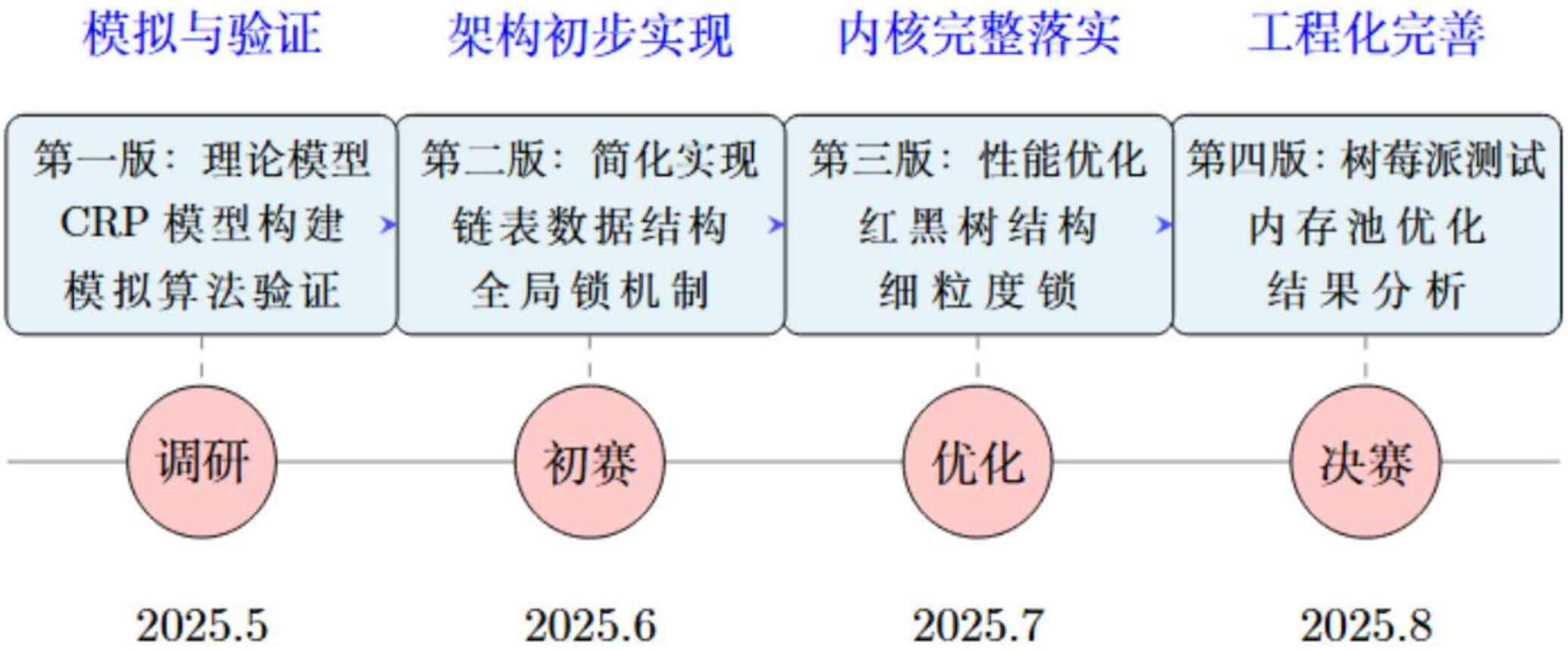
- **轻量化设计和低时延**：简化数据结构和高效调度算法。
- **关键任务优先处理**：伪抢占优先调度关键任务。
- **缓存亲和性优化**：基于 CRP(cache recency profile) 缓存感知机制的五级调度策略，最大化缓存利用率。
- **模块化和兼容性**：独立调度类与数据结构，与现有调度器并存，渐进式部署。

2 项目介绍

PROJECT INTRODUCTION

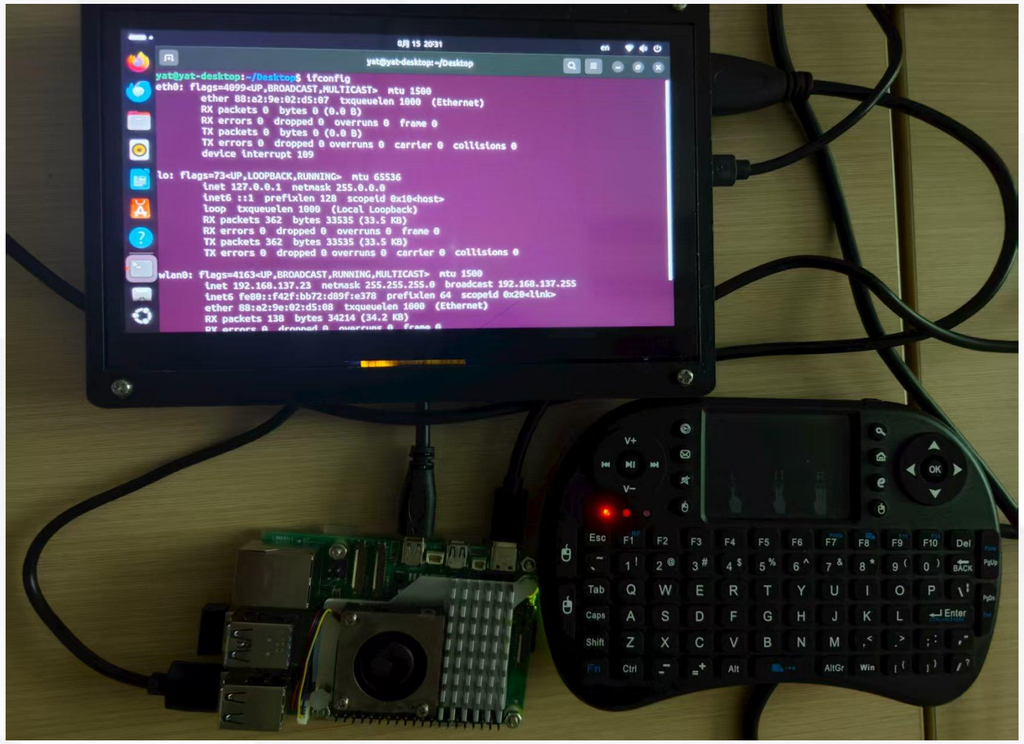


项目开发历程


模拟与验证
架构初步实现
内核完整落实
工程化完善
第一版：理论模型
CRP 模型构建
模拟算法验证
第二版：简化实现
链表数据结构
全局锁机制
第三版：性能优化
红黑树结构
细粒度锁
第四版：树莓派测试
内存池优化
结果分析
调研
初赛
优化
决赛
2025.5
2025.6
2025.7
2025.8

项目平台

VMware 搭建整体内核框架（x86）--->Qemu 仿真测试--->树莓派实机测试（arm）



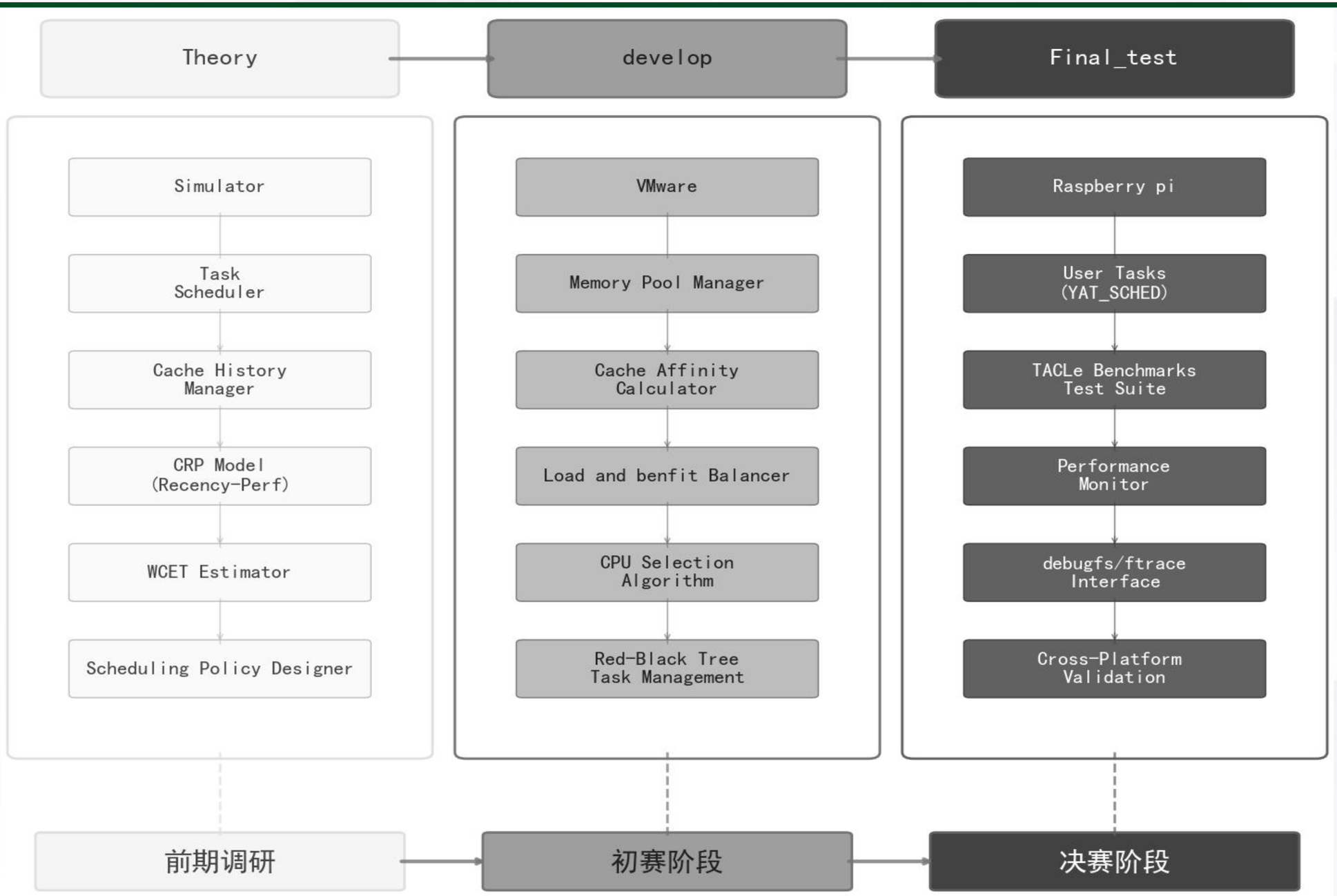
项目完成情况

赛题行动项	完成情况
【行动项1】 设计轻量化内核调度类，替代CFS复杂机制	实现SCHED_YAT_CASCHED调度策略，精简数据结构，**降低调度开销**。
【行动项2】 支持关键进程优先调度与抢占	实现**伪抢占，优先级高的任务优先**，关键任务可抢占普通任务。
【行动项3】缓存感知，实现缓存感知亲和性	维护多级缓存历史表，基于 CRP(cache recency profile) **缓存感知机制的五级调度策略**，最大化缓存利用率。
【行动项4】 模块化和兼容性，支持不同平台架构。	**独立调度类**，与现有调度器并存，渐进式部署。**在x86和arm架构下均实现部署**。

具体开发过程

项目步步推进
历经以下平台
开发和测试：

- 理论探究
- 模拟验证
- VMware搭建整体内核框架（x86）
- Qemu仿真测试
- 树莓派实机测试（arm）

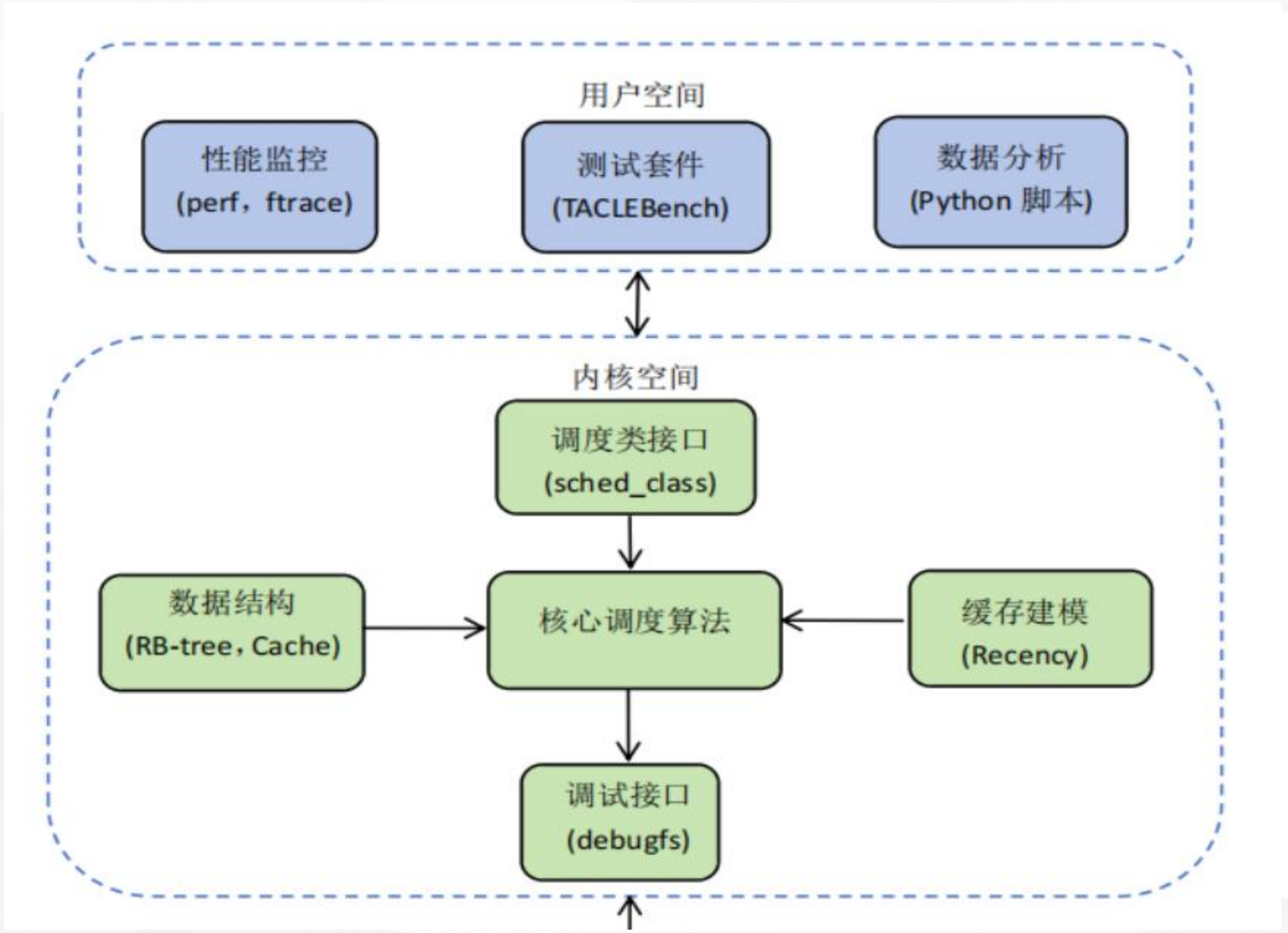


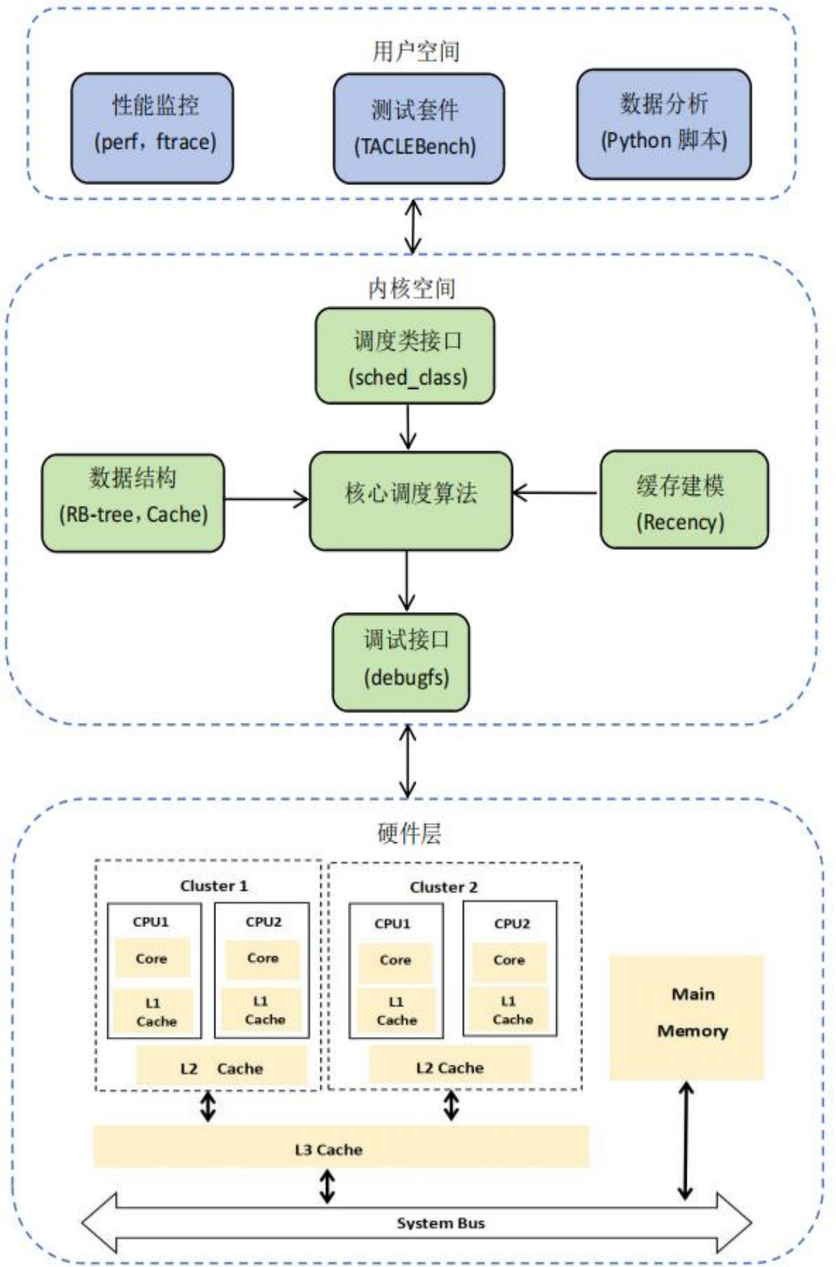
3 总体架构

OVERALL ARCHITECTURE



三层架构设计


用户空间
性能监控
(perf，ftrace)
测试套件
(TACLEBench)
数据分析
(Python 脚本)
内核空间
调度类接口
(sched_class)
数据结构
(RB-tree，Cache)
核心调度算法
缓存建模
(Recency)
调试接口
(debugfs)


用户空间
性能监控
(perf，ftrace)
测试套件
(TACLEBench)
数据分析
(Python 脚本)
内核空间
调度类接口
(sched_class)
数据结构
(RB-tree，Cache)
核心调度算法
缓存建模
(Recency)
调试接口
(debugfs)
硬件层
Cluster 1
Cluster 2
CPU1
CPU2
Core
L1 Cache
L2 Cache
L3 Cache
Main Memory
System Bus

CSCC 全国大学生计算机系统能力大赛

中山大學 SUN YAT-SEN UNIVERSITY

4 设计与实现

DESIGN & IMPLEMENTATION



调度策略概述

针对现有调度器(CFS)的不足，我们的改进Yat-CAshed策略：

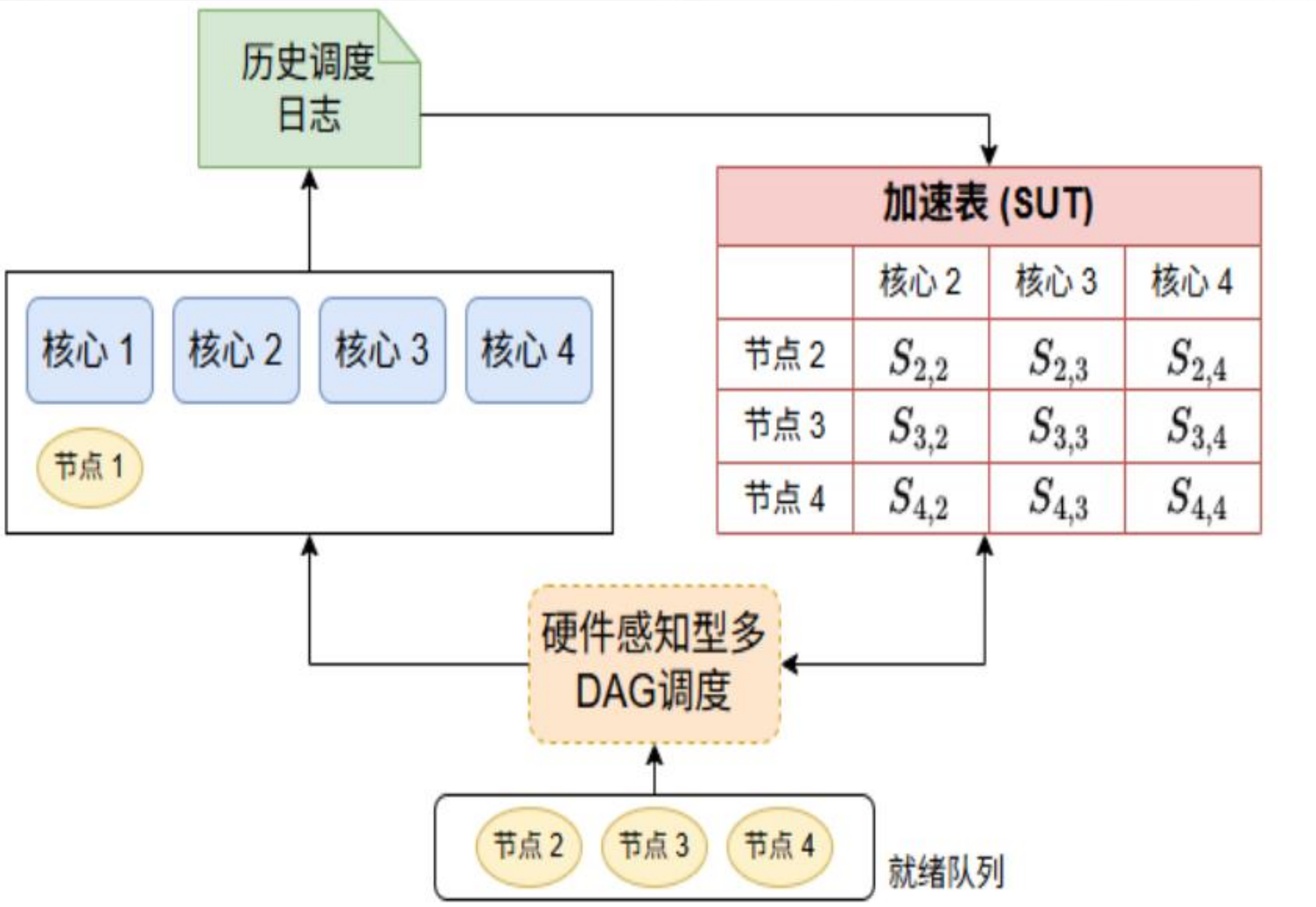
1. **维护历史信息**：记录每个任务在各CPU上的最近运行时间（recency）
2. **缓存感知调度**：根据recency优先选择缓存热度高的CPU，提升缓存命中率
3. **分层决策与分层队列**：结合任务优先级和WCET进行分层排序，兼顾关键任务优先与系统响应
4. **智能平衡**：在缓存利用率和负载均衡之间动态权衡，减少不必要的迁移

```
|—调度策略常量定义
    ├——— SCHED_NORMAL (0)
    ├——— SCHED_FIFO (1)
    ├——— SCHED_RR (2)
    ├——— SCHED_BATCH (3)
    ├——— SCHED_IDLE (5)
    ├——— SCHED_DEADLINE (6)
    └——— SCHED_YAT_CASCHED (8) (new)
```

Yat
Cache-Aware Shed

全局调度框架

- ***任务接收***：调度器接收来自就绪队列的多个任务节点。
- ***调度决策***：硬件感知调度器综合考虑加速表信息、核心负载状态和缓存亲和性，做出最优分配决策。
- ***执行监控***：任务在指定核心上执行，系统收集执行性能数据。
- ***历史查询***：查询历史调度日志，获取相关任务的历史执行数据。



全局调度框架示意图

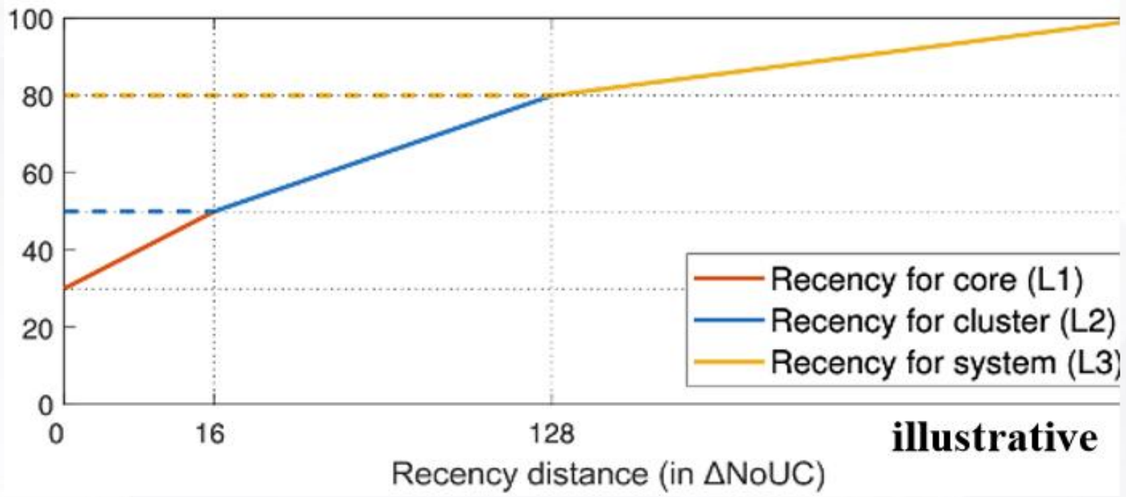
CRP（Cache Recency Profile）简介

缓存重用距离是 Yat-CAshed 算法的核心概念，定义为：

$$r(u_j, L_x) = \Delta NoUC(u_{j,v}, u_{j,v-1}, L_x)$$

其中：

- L_x：第 x 级缓存
- $u_{j,v}$：节点 u_j 的第 v 次作业实例
- $\Delta NoUC(\cdot)$：两次连续访问之间其他任务访问的唯一缓存行数量



number of unique cache lines (NOUC)

CRP（Cache Recency Profile），***缓存重用距离模型***，是一种**轻量化**任务缓存局部性的经验模型。

核心思想：通过度量任务自上次在某CPU或集群上运行以来，期间其他任务对该缓存层的占用，量化当前任务在该CPU上命中缓存的概率。缓存重用距离越短，任务重用缓存数据的可能性越高，执行效率越好；距离越长，缓存失效概率越高，执行时间趋近于最坏情况。

recency distance = 两次目标任务之间，所有干扰任务的NOUC之和

CRP为调度器提供了缓存感知的决策依据，帮助任务优先分配到“缓存更热”的CPU，提升系统性能。

CRP简介

由于新指令数量通常是时间的线性递增函数，缓存重用距离可以通过以下公式近似：

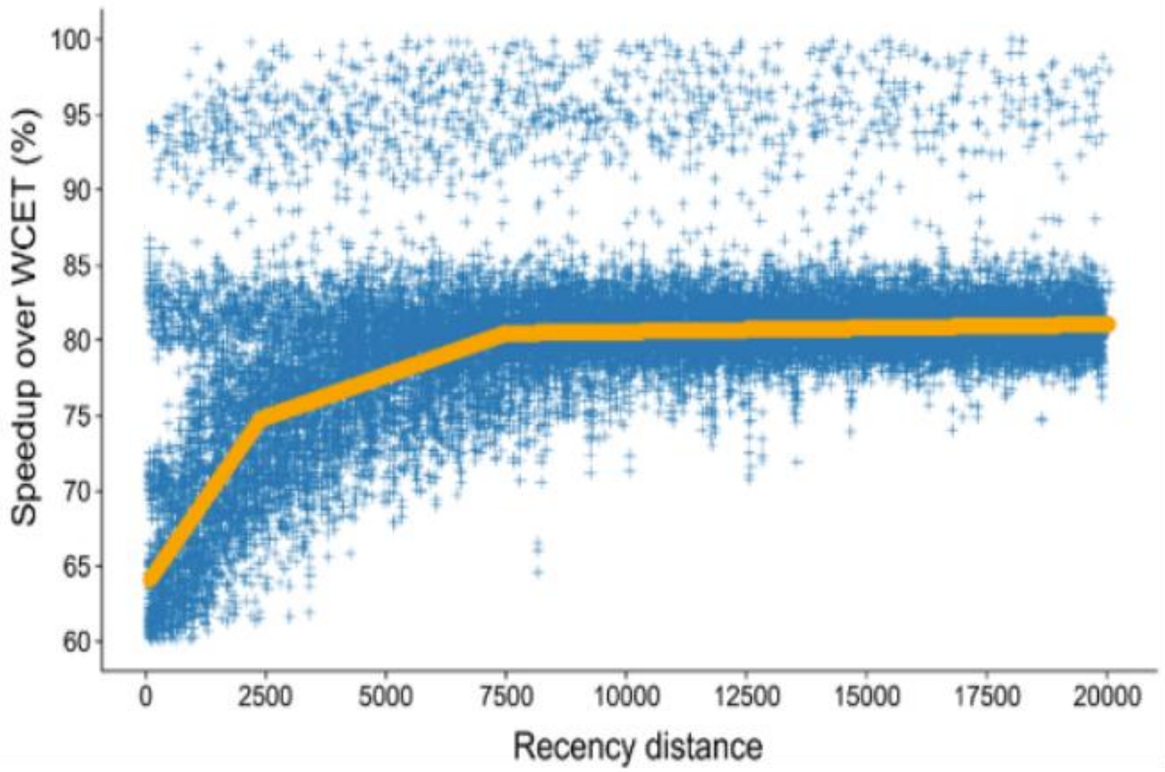
$$r(u_j, L_x) = g(t(u_{j,v}) - t(u_{j,v-1}), L_x)$$

其中 $g(\cdot)$ 函数表示在时间间隔内其他任务执行时间的总和。

CRP模型如何获得？

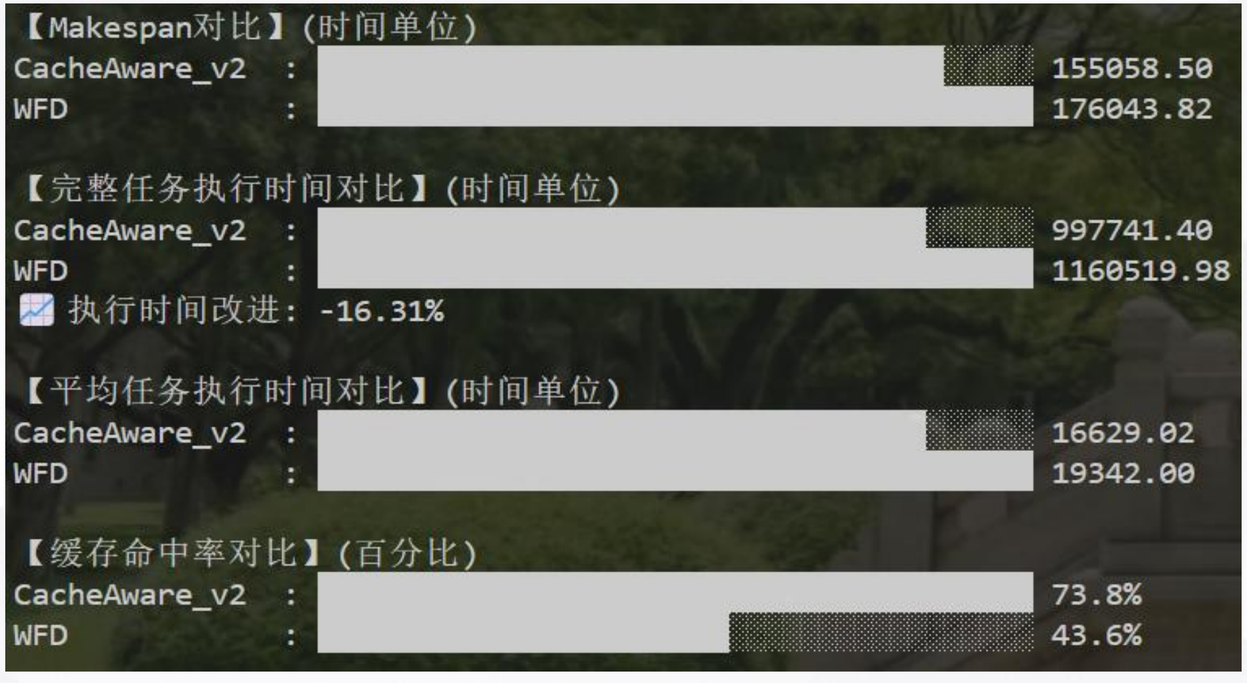
- **实验平台**：树莓派等真实多核硬件
- **实验方法**：
  - 多次运行同一任务，每次运行之间插入不同量的干扰任务，控制recency distance（缓存重用距离）。
  - 记录每次运行的实际执行时间，并与最坏执行时间（WCET）做比值。
  - 横轴为recency distance，纵轴为speedup over WCET（加速比）。
- **数据处理**：采用分段线性回归拟合实验数据，得到CRP主趋势曲线。

CRP建模拟合图



调度模拟器验证缓存的影响

- 比较缓存敏感的Yat-CAshed简易调度策略和缓存不敏感的WFD调度策略
- 反映缓存感知调度在实际多核系统中的性能表现，验证了缓存在调度和任务执行过程中的重要性。




```
Yat-CAShed/
├── allocation/                  # 调度算法模块
│   ├── AllocationMethods.java      # 算法基类
│   ├── OnlineWFD.java              # WFD算法实现
│   └── OnlineCacheAware.java       # Cache-Aware算法实现
├── analyzer/                    # 性能分析模块
│   └── PerformanceAnalyzer.java    # 性能指标计算和分析
├── entity/                      # 核心实体模块
│   ├── Node.java                   # 任务节点定义
│   └── RecencyProfileReal.java     # 缓存行为建模
├── generator/                   # 任务生成模块
│   ├── EnhancedTaskGenerator.java # 增强任务生成器
│   ├── CacheHierarchy.java         # 缓存层次建模
│   └── UUnifastDiscard.java        # 均匀分布任务生成
├── parameters/                  # 系统参数模块
│   └── SystemParameters.java       # 全局配置参数
├── visualizer/                  # 可视化模块
│   ├── ResultVisualizer.java       # Java可视化器
│   ├── simple_visualize_results.py # Python简化可视化
│   ├── visualize_results.py        # Python完整可视化
│   ├── requirements.txt            # Python依赖
│   └── result/                     # 结果输出目录
├── lib/                         # 第三方库目录
├── classes/                     # 编译输出目录
├── EnhancedComparisonExperiment.java # 主实验程序
├── LibraryVerification.java     # 库依赖验证
├── QuickVerificationTest.java   # 快速验证测试
├── Makefile                     # Unix构建脚本
├── build.bat                    # Windows构建脚本
└── Simulator.md                 # 项目说明文档
```

负载计算

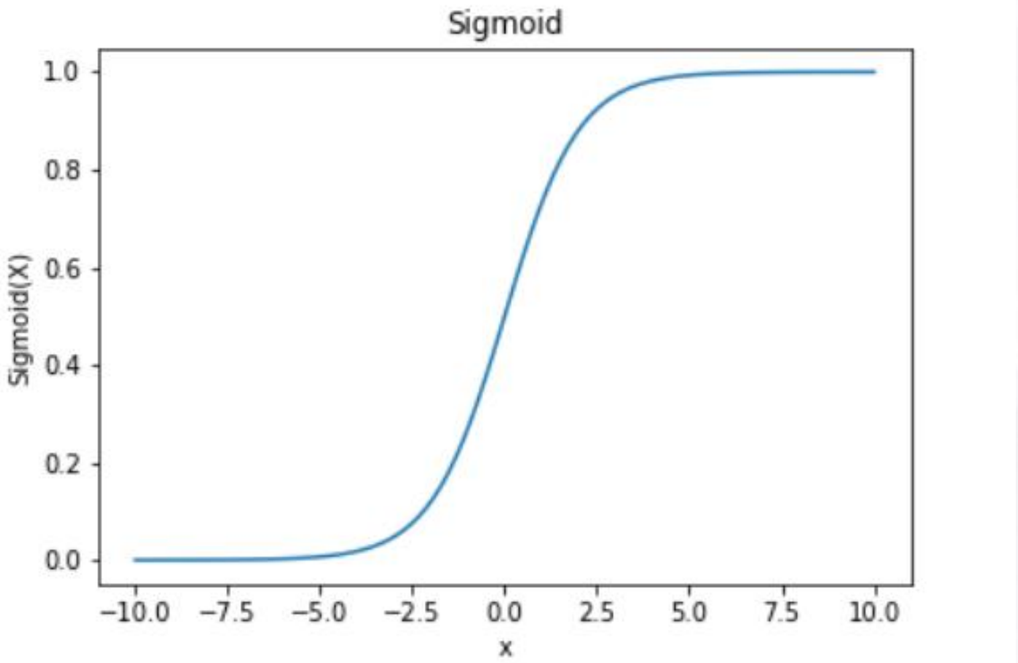
负载计算函数 calculate_cpu_load 通过遍历红黑树累加所有任务的 WCET：

$$Load_{cpu} = \sum_{task \in RB_Tree_{cpu}} WCET_{task} + WCET_{current}$$

- 在每个yat_casched_rq结构体中增加一个u64 load成员，专门用于记录该CPU当前Yat_Casched任务的WCET总和。
- 每次有任务入队（enqueue）或出队（dequeue）时，直接增减该任务的wcet到load变量，这样查询负载时只需O(1)读取
- **用途：**这个负载值用于调度决策，比如在无空闲CPU或者上次运行cpu不存在，则选择此时负载最小的那个来分配新任务，从而实现负载均衡。

关键任务优先执行机制

- 通过人为定义优先级区间和key计算方式，可以保证关键任务优先。key越小，任务在红黑树越靠前，当调用pick_next_task时优先被调度。这种方法简单高效，易于扩展和维护。



- 普通任务（prio 100~139）：key = prio + sigmoid(wcet)（prio越小越优先）
- 实时任务（prio 0~99）：key = prio

这样也就支持了对指定数量关键进程的优先调度，只需要系统调用设置好任务的优先级和wcet，这些线程就可以抢占其他非关键进程（优先级较低）。

调度策略流程

- **策略1** 首次运行任务从未执行过

  → 全局负载最低 CPU。

- **策略2** 热缓存复用上次跑的 CPU

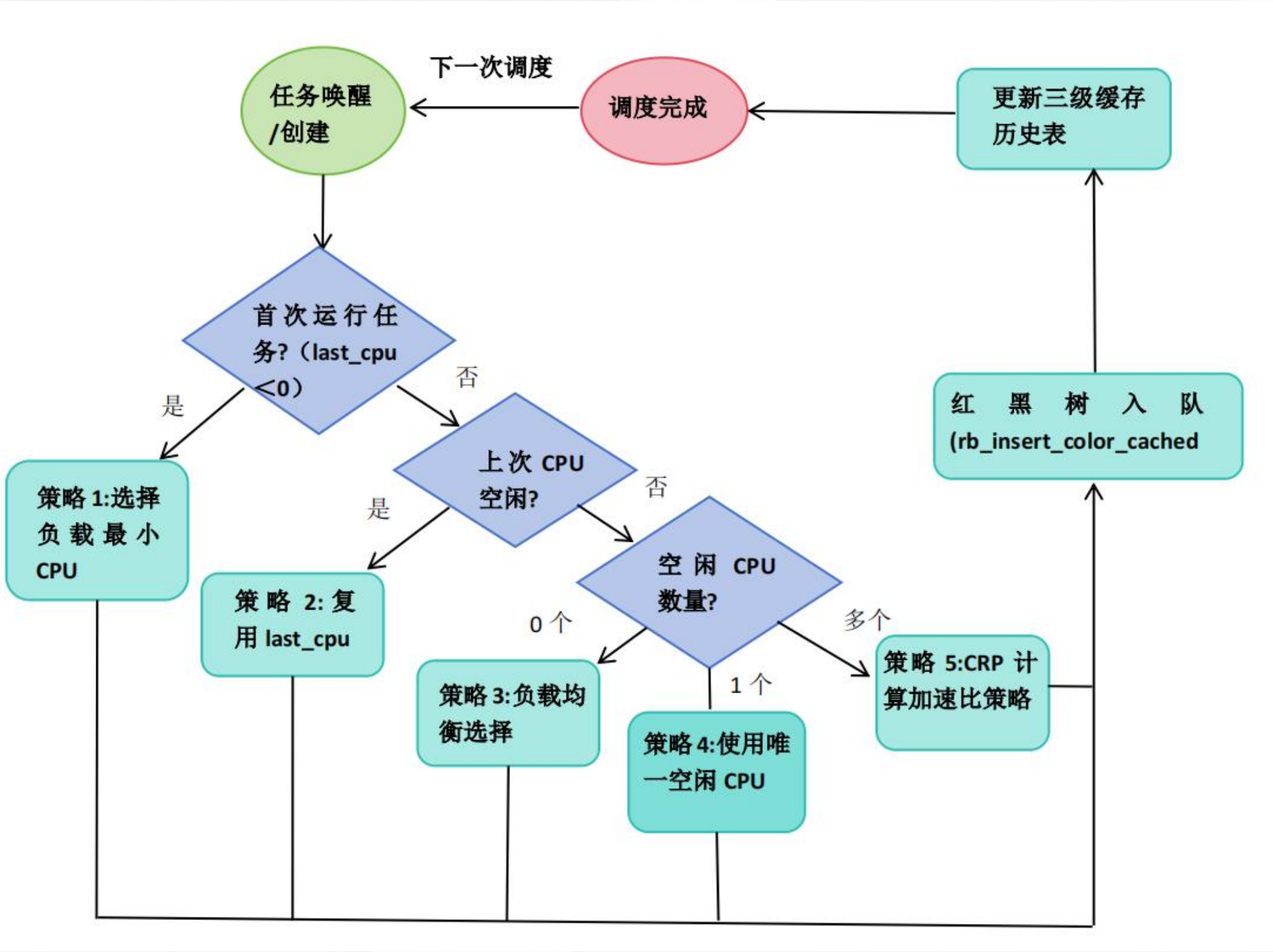
  正好空闲 → 继续使用，零迁移开销。

- **策略3** 唯一空闲只剩 1 个空闲核

  → 没的选，立刻上。

- **策略4** 全忙负载均衡无空闲核

  → 在“忙碌核”中选负载最低者。



- **策略5** 多空闲择优≥2 个空闲核 → 计算缓存重用距离，挑最大加速比核，提升命中

调度算法伪代码

算法 1: Yat-CAShed 在线分配算法（五策略内核实现版）

Input: task p, available CPUs Λ^*, History H, CRP model

Step 1: CPU 选择策略判断;

if $p.last_cpu < 0$ **or** $!cpu_online(p.last_cpu)$ **then**
  ; // 策略 1: 首次运行任务: 负载均衡
  $best_cpu \leftarrow -1$, $min_load \leftarrow ULLONG_MAX$;
  for $cpu_i \in online_cpus$ **do**
    $load \leftarrow calculate_cpu_load(cpu_i)$;
    if $load < min_load$ **then**
      $min_load \leftarrow load$, $best_cpu \leftarrow cpu_i$;
    end
  end
  return $best_cpu$;
else
  $last_cpu \leftarrow p.last_cpu$;
  $idle_cpus[] \leftarrow$ 统计空闲 CPU 列表;
  if $last_cpu$ 在 $idle_cpus$ 中 **then**
    ; // 策略 2: 上次运行 CPU 空闲，直接复用
    return $last_cpu$;
  else
    if $|idle_cpus| = 1$ **then**
      ; // 策略 3: 仅 1 个空闲 CPU
      return $idle_cpus[0]$;
    else
      if $|idle_cpus| = 0$ **then**
        ; // 策略 4: 0 个空闲 CPU，负载均衡（代码与策略 1 相同）
      else
        ; // 策略 5: 多个空闲 CPU，按加速比选择
        $best_cpu \leftarrow -1$, $max_benefit \leftarrow 0$;
        for $cpu_i \in idle_cpus$ **do**
          $recency \leftarrow calculate_recency(p, cpu_i)$;
          $benefit \leftarrow estimate_acceleration(p, cpu_i, recency)$;
          if $benefit > max_benefit$ **then**
            $max_benefit \leftarrow benefit$, $best_cpu \leftarrow cpu_i$;
          end
        end
        return $best_cpu$;
      end
    end
  end
end

Step 2: 历史记录更新，便于计算 recency 和加速比;

$add_history_record(selected_cpu, p, exec_time)$;

CSCC 全国大学生计算机系统能力大赛

中山大學 SUN YAT-SEN UNIVERSITY

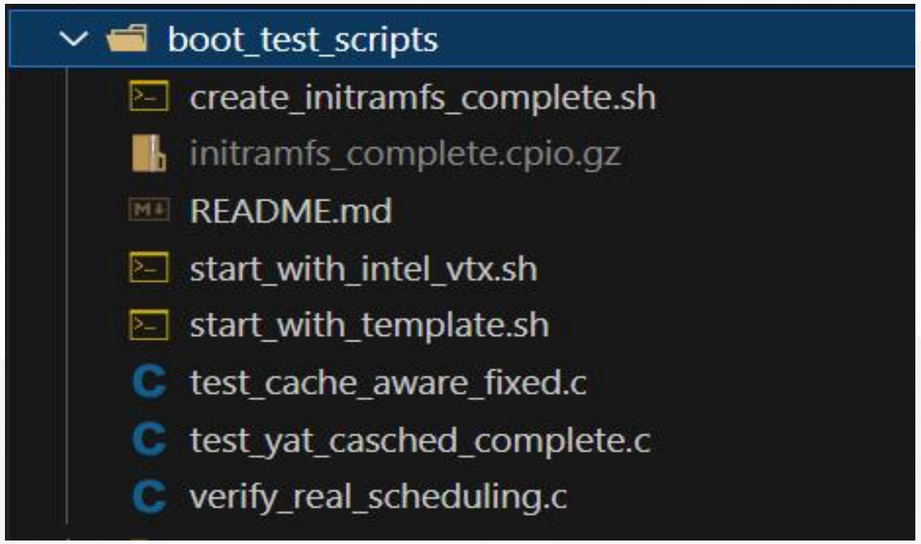
5 项目测试

PROJECT TESTING



项目测试结构

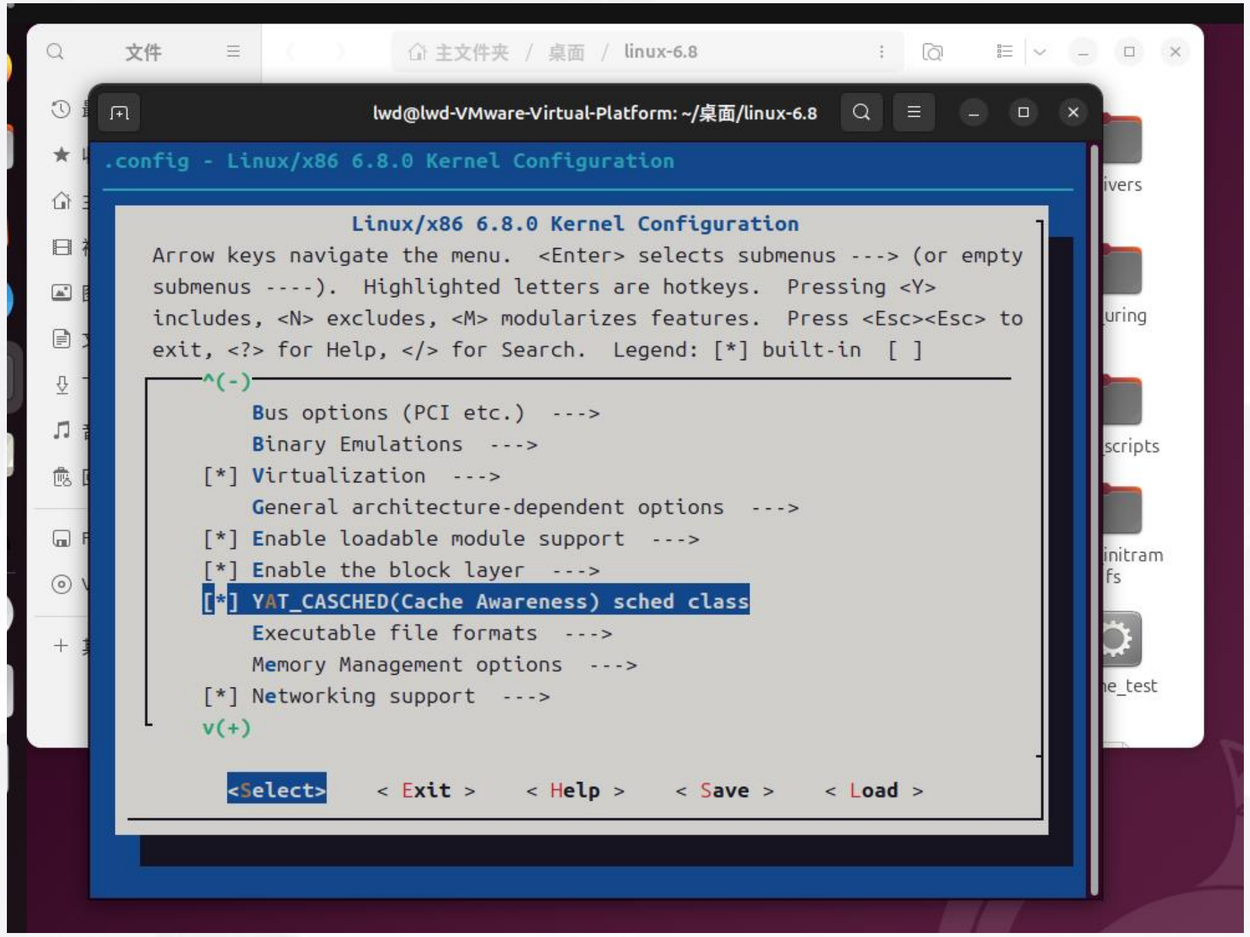
- 启动测试与功能验证
- 模拟任务集测试
- 优先级调度验证
- TacleBench测试集
- 自动化工具



测试框架架构

```
yat_test/
    基础功能验证
        verify_real_scheduling.c      # 调度器集成验证
        system_call_test.c            # 系统调用接口测试
    模拟任务集测试
        CFS_test.c                    # CFS调度器模拟任务集
        yat_simple_test.c             # YAT调度器模拟任务集
    优先级调度验证
        priority_test.c               # 关键进程优先测试
    基准测试集
        tacle_kernel_test.c           # TacleBench真实任务集对比
    自动化工具
        build_CFS_simple_test.sh      # CFS测试环境构建
        start_CFS_simple_test.sh      # CFS QEMU测试启动
        build_yat_simple_test.sh      # YAT测试环境构建
        start_yat_simple_test.sh      # YAT QEMU测试启动
        build_priority_test.sh        # 优先级测试环境构建
        start_priority_test.sh        # 优先级测试启动
        build_tacle_kernel_test.sh    # TacleBench测试构建
        start_tacle_kernel_test.sh    # TacleBench测试启动
        analyze_results.py            # 结果分析脚本
```

启动测试与功能验证


文件
主文件夹 / 桌面 / linux-6.8
lwd@lwd-VMware-Virtual-Platform: ~/桌面/linux-6.8
.config - Linux/x86 6.8.0 Kernel Configuration
Linux/x86 6.8.0 Kernel Configuration
Arrow keys navigate the menu. <Enter> selects submenus ---> (or empty submenus ----). Highlighted letters are hotkeys. Pressing <Y> includes, <N> excludes, <M> modularizes features. Press <Esc><Esc> to exit, <?> for Help, </> for Search. Legend: [*] built-in [ ]
^(-)
Bus options (PCI etc.) --->
Binary Emulations --->
[*] Virtualization --->
General architecture-dependent options --->
[*] Enable loadable module support --->
[*] Enable the block layer --->
[*] YAT_CASCHED(Cache Awareness) sched class
Executable file formats --->
Memory Management options --->
[*] Networking support --->
v(+)
<Select> < Exit > < Help > < Save > < Load >


最近
收藏
主文件夹
视频
图片
文档
下载
音乐
回收站
Floppy Disk
VMware Tools
其它位置
终端
=== 多线程测试 ===
Thread 1: TID=151 started
✓ Successfully set Yat_Casched policy for PID 151
Thread 1: ✓ 成功设置 Yat_Casched 策略
Thread 2: TID=152 started
✓ Successfully set Yat_Casched policy for PID 152
Thread 2: ✓ 成功设置 Yat_Casched 策略
Thread 4: TID=154 started
✓ Successfully set Yat_Casched policy for PID 154
Thread 4: ✓ 成功设置 Yat_Casched 策略
Thread 3: TID=153 started
✓ Successfully set Yat_Casched policy for PID 153
Thread 3: ✓ 成功设置 Yat_Casched 策略
Thread 1: CPU切换次数: 0
Thread 2: CPU切换次数: 0
✓ 线程 1 成功完成
✓ 线程 2 成功完成
Thread 4: CPU切换次数: 0
Thread 3: CPU切换次数: 0
✓ 线程 3 成功完成
✓ 线程 4 成功完成
多线程测试结果: 4/4 成功
=== 性能测试 ===
✓ Successfully set Yat_Casched policy for PID 146
计算任务完成时间: 1725314 ns (1.73 ms)
结果检查: sum = 16866920716384
Yat_Casched 测试报告
总测试项目: 10
成功项目: 10
失败项目: 0
成功率: 100.0%
Yat_Casched 调度器工作正常!
建议的后续测试:
- 查看 /proc/sched_debug 获取详细调度信息
- 使用 'ps -eo pid,class,comm' 查看进程调度类
- 检查内核日志中的调度器相关信息
~ #

模拟任务集测试

模拟任务集测试配置

```
// 模拟任务集测试参数
#define SCHED_CFS 8 // CFS调度策略
#define NUM_ROUNDS 100 // 测试轮数
#define TASKS_PER_ROUND 12 // 每轮任务数
#define CYCLES_PER_TASK 50 // 每个任务的周期数
#define STANDARD_PRIORITY 120 // 统一优先级

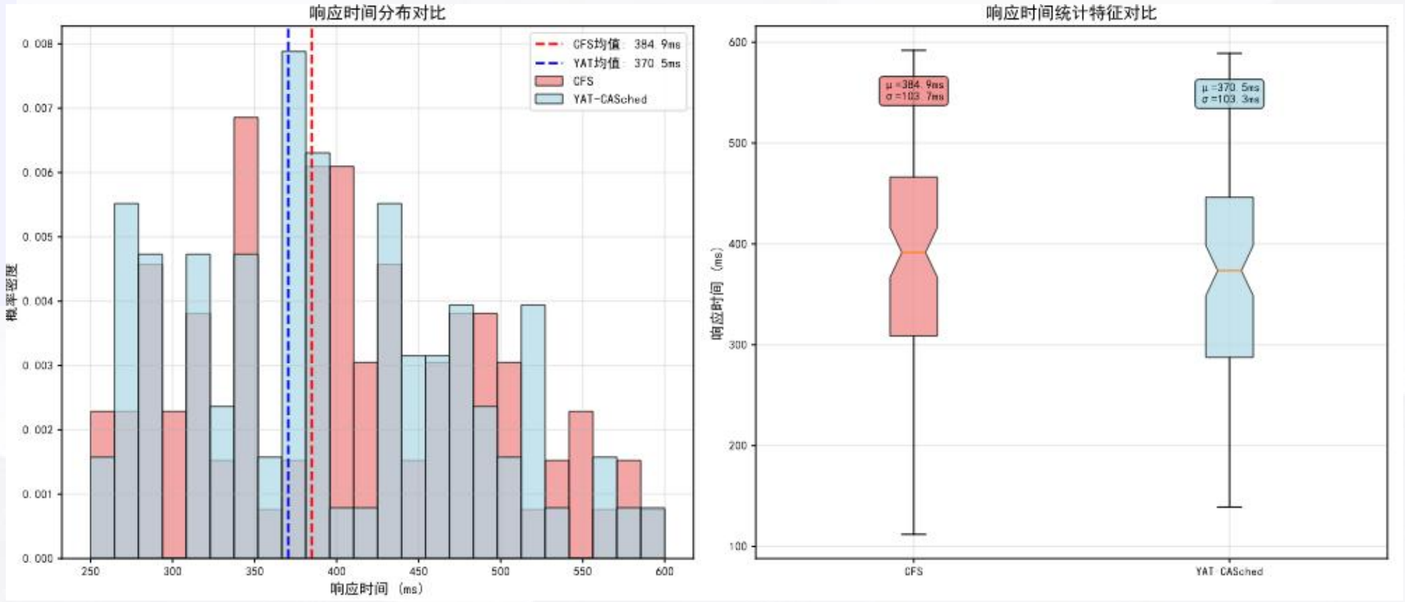
// CFS_test.c: 使用Linux CFS调度器
// yat_simple_test.c: 使用YAT-CASched调度器
```

YAT-CASched表现出更加集中的执行时间分布和更低的平均值

模拟任务集执行时间对比表

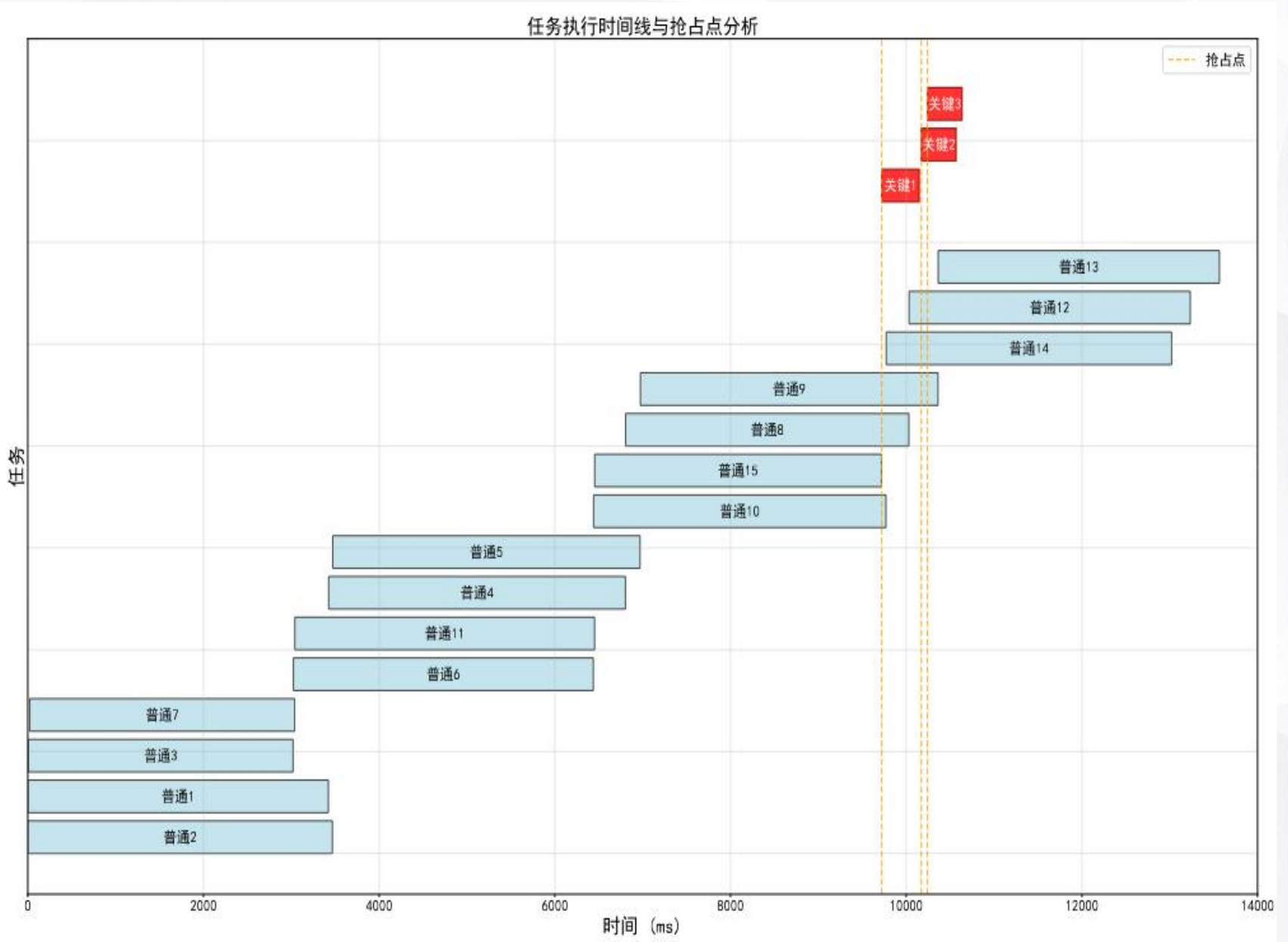
性能指标	YAT-CASched	CFS	改善幅度
平均执行时间 (ms)	370.51	384.85	+3.88%
最大执行时间 (ms)	589.00	592.00	–
最小执行时间 (ms)	139.00	112.00	–
执行时间标准差	103.25	103.70	+0.44%
任务完成率 (%)	100.0	100.0	–
平均调度延迟 (s)	0.6175	0.6414	+3.87%

模拟任务集执行时间分布对比图



优先级调度验证

- 如图，通过时间戳分析展示了伪抢占行为的详细情况。
- **在关键任务3被创建后，抢占了执行任务12或14的核心**
- 从而验证了调度器的关键进程优先运行机制工作正常。



基准测试集总时间对比

表 11: TacleBench 真实任务集性能对比（12 任务 ×50 副本 ×10 轮）

性能指标	CFS 调度器	YAT-CASched	改善幅度
平均轮次用时 (ms)	1550.8	1413.1	+9.7665%
总执行时间 (s)	15508.0	14131.0	+9.7665%

- 真实任务集缓存优化效果：基于 TacleBench 基准测试的结果显著，YAT-CASched 实现了 **9.77%** 的整体执行时间优化。
- 这一结果直接验证了缓存感知调度算法在真实计算密集型任务中的核心价值。

```
=== 第8轮 ===
本轮实际用时: 2030 ms
=== 第9轮 ===
本轮实际用时: 1689 ms
=== 第10轮 ===
[   16.379540] hrtimer: interrupt took 8649740 ns
本轮实际用时: 1436 ms
=== 所有TacleBench Kernel任务完成 ===
平均每轮用时: 1550.8 ms
总进程数: 12000 个
=== 所有测试任务完成，进入shell ===
```

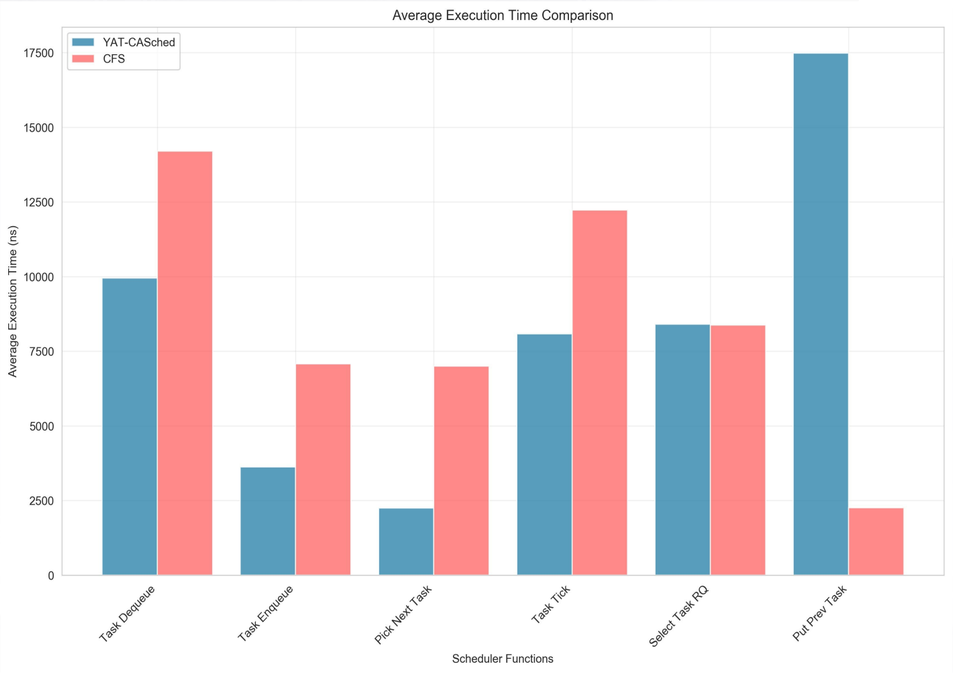
```
=== 第8轮 ===
本轮实际用时: 1324 ms
=== 第9轮 ===
本轮实际用时: 1380 ms
=== 第10轮 ===
本轮实际用时: 1362 ms
=== 所有TacleBench Kernel任务完成 ===
平均每轮用时: 1413.1 ms
总进程数: 12000 个
=== 所有测试任务完成，进入shell ===
```

基准测试集调度开销对比

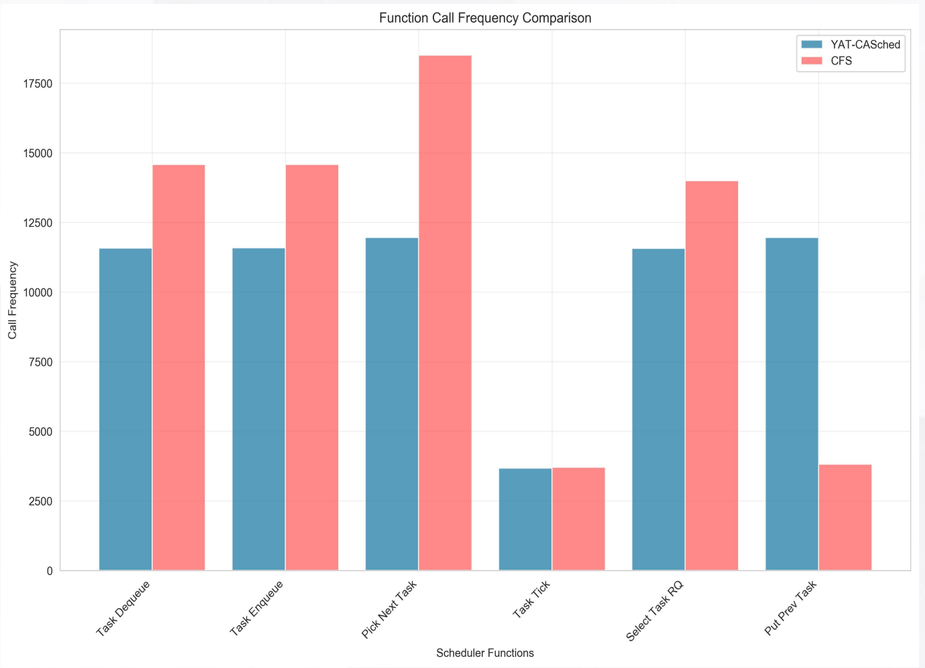
- Yat-CAShed调度开销小于 CFS，总执行时间减小 **14.9%**，减小验证了调度器轻量化的实现。

调度函数总执行时间对比（基于 100 轮的模拟任务集调度统计）

调度器	YAT-CAShced	CFS
核心函数总调用次数	62311	69163
平均执行时间（ns）	8297.85	8522.74
总执行时间（ms）	519.90	610.75



函数平均执行时间对比图



函数平均执行次数对比图

6 总结与展望

CONCLUSION & OUTLOOK



总结与创新性工作

- 本项目成功设计并实现了Yat-CASched缓存感知调度器，有效解决了CFS调度器臃肿、缺乏关键线程优先调度、时延不可控等核心问题。
- 基于Cache Recency Profile等理论，构建了分层调度架构，实现了轻量化设计，低延迟保证以及性能优化。
- 通过完整的"理论验证→模拟验证→内核集成→性能测试"技术路线，成功将调度器集成到Linux 6.8内核。

```
kernel/sched/
├── yat_casched.c      # 主调度器实现（核心）
├── sched.h            # 调度器头文件 (修改)
└── core.c             # 内核调度核心 (修改)

include/linux/sched/
├── yat_casched.h      # 调度器头文件(新增)
└── task.h             # 任务结构体定义 (修改)

include/uapi/linux/
└── sched.h            # 用户空间调度定义(修改)
```

🎯 创新技术	📋 技术描述
五策略智能调度算法	基于系统状态进行差异化CPU选择决策
三层缓存历史表架构	L1/L2/L3缓存精确建模，支持微秒级调度决策
CRP缓存收益模型	Cache Recency Profile理论的缓存重用距离计算
内存池+红黑树优化	微秒级调度延迟，生产级性能表现

展望

Yat-CAShed 项目是团队一次宝贵的内核编程实践。

同时我们清醒地认识到，本次工作仅是学习操作系统万里长征的第一步。未来我们将以更大的热忱投入到操作系统的学习研究中。



THANKS



望各位评委老师们批评指正！